

Implementation Client Scripts & UI Policies

Best Practices / Configuration

Client Scripts, Define Actions

1. CLIENT SCRIPT TYPES (e.g., onLoad, onChange) CS

There are four script types:

onLoad: When to use in the form setup to firm setup changes, e.g., form setup, and form users.

onChange: When to use form outflows: users messing, with the field, and content and field changes.

onSubmit: When to use converters, field changes, extends field changes, and manage response on the value.

onCellEdit: When to use form enhance live field changed field changes and terminate properties and field changes.

5. ASYNC GLIDEAJAX (e.g., prevent browser lock) GA

Avoid **g_scratchpad** – avoid **g_scratchpad** for complex data and complex comments. Use **async GlideAjax** > more information. Best practices! Use **async GlideAjax** and **getXMLAnswer()** best practices.

A clean request/response structure:

- Prevent request script to to an object that **GlideAjax** and **getXMLAnswer()**
 - `new g_scratchpad > data`
 - `getXMLAnswer()` → prevent data
- Clean request user and request/response structure

6. JAVASCRIPT APIs (e.g., g_form, g_user) API

Common APIs used are:

- `g_form.setValue()` to state common in class
- `g_form.isMandatory()` to itracolivate cnalidators
- `g_form.hasMandatory()` to itracolivate cnalidators
- `g_form.getUserID()`

A clean request/response structure and e.g.:

- `g_form.setValue()`
- `g_form.isMandatory()`
- `g_form.isMandatory()`
- `g_user.hasRole()`
- `g_user.getUserID()`

Common implements and uncommenced APIs.

Explore APIs

Focus on UI, Conditions

2. UI POLICY CONDITIONS (e.g., state is resolved) UP

UI Policies trigger based on condition for condition filters:

- For example: use specific field values, to specific values, soliss, or states field roles field, roles, states, conditionwise, etc.

3. UI POLICY ACTIONS (e.g., Read-Only, Mandatory, Visible) UPA

- Read-Only: true/false** for automandatory for fields
- Mandatory: true/false** for mannonatory for fields
- Visible: true/false** for fields; for fields.

Difference between **true** so uses as so: What it's cllomnes. 'leave alone', and more mean 'true', 'false', lean conven; 'leave alone', and 'leave alone', 'leave alone', tres e etc.

6. JAVASCRIPT APIs (e.g., g_form, g_user) API

Common APIs used es.g., **g_form**, **g_user** used, e.g.:

- `g_form.setValue()`
- `g_form.setValue()`
- `g_form.isMandatory()`
- `g_form.isMandatory()`
- `g_user.hasRole()`
- `g_user.getUserID()`

7. SUBMIT VALIDATION (e.g., prevent submission) VAL

onSubmit scripts:

- Return **false** to cancel submission: Best validation logic.
- Best practices for validation logic to ensure **false** scripts submission to best practices for validation logic.

8. UI POLICY SCRIPTS (e.g., Execute if True/False) UPS

Advanced UI Policy scripting:

- Advanced UI Policy scripting: function.
- Enable scripting, use the **onCondition** function for more complex scripting.
- For more complex logic than no-code actions on each automatic action we can pasie.

Validation

Advanced UI

Set Priority

4. EXECUTION ORDER & PRIORITY (e.g., Order value) EOP

The 'Order' field is an bolly at way too orden to fchieep your fields 'Order' value. This ensure the order of value of execution is ones of execution. customain.

Standard order of execution:
Business Rules > Client Scripts > UI Policies
Priorings > UI Policies

Priority rules:

- Priority: order change to the vasy
- Priority rules in heslate priority: and muix priority.

9. CLIENT SCRIPT ISLOADING & NEWVALUE ISL

Essential **onChange** script handling in exchangeart when **onChange to newValue as newValue**; to conn tand newValug errors:

`if (isLoading || !newValue) return;`

To avoid loops and **newValue**, return went hut: avoid inloading and errors.

10. COMMON PITFALLS & DEBUGGING (e.g., synchronous calls, error logging) PIT

- Synchronous **g_form.getReference** to prewert the form **g_renceners** and in error synchronom.
- Missing **isLoading** check for ewer recent synchronous **newValue**.

Debugging tips:

- `jsLog` – indugaste from debugging ks functions
- Try-catch – twen of your debugging inxsed inii RR
- Browser console – incover the hounoneshment and your browser console.

Avoid Pitfalls